

Lab07 - Block Ciphers - AES and block cipher modes

Part 01 - Encrypt - GCM (current)

In this lab, you will explore the practical implementation of AES (Advanced Encryption Standard) encryption and decryption. The primary focus of this lab is for you to implement key cryptographic functions within the client, specifically AES encryption in GCM mode, and AES decryption. You will also look at an older cipherblock mode to see the problems it had.

1. To get the skeleton code for all parts of the lab:

```
scp -r repo@repo.homework.3310.com:/home/repo/lab07 .
```

```
cpre3310@cpre3310:~$ scp -r repo@repo.homework.3310.com:/home/repo/lab07 .
repo@repo.homework.3310.com's password:
part03_input_bmp.bmp          100% 703KB  4.6MB/s  00:00
part03_ecb_cbc_skel.py        100% 3366  989.2KB/s 00:00
part01_key.bin                 100% 16    4.7KB/s  00:00
part01_nonce.bin               100% 12    2.9KB/s  00:00
part01_02_aes_student_skel.py 100% 4775  1.9MB/s  00:00
part01_plaintext.txt           100% 91    27.9KB/s 00:00
part04_padding_oracle_demo.py 100% 8582  2.0MB/s  00:00
part02_nonce.bin               100% 12    5.5KB/s  00:00
part02_ciphertext.bin          100% 133   81.9KB/s 00:00
part02_key.bin                 100% 16    5.5KB/s  00:00
```

Remember you will have to give yourself permissions to edit the files.

2. You will see TODO sections in the code. You can determine how to complete the calls by reading <https://cryptography.io/en/latest/hazmat/primitives/aead/#cryptography.hazmat.primitives.ciphers.aead.AESGCM> .
 - a. Use AESGCM, 128 bit keys
3. The syntax to run your encryption program is `python3`
`part01_02_aes_student_skel.py encrypt part01_key.bin`
`part01_nonce.bin part01_plaintext.txt part01_ciphertext.bin`

```
jrursch@ames-desktop-2:/mnt/3310_labs/labs/f25_labs/lab07/part01$ python3 part01_aes_student_skel_jar.py encrypt part01_key.bin part01_nonce.bin part01_plaintext.txt part01_ciphertext.bin
File 'part01_plaintext.txt' encrypted successfully to 'part01_ciphertext.bin'.
```

4. **Print the part01_ciphertext.bin file and include a screenshot of the result in your lab report.** Use `xxd -b <outputFile>` to display both the binary and ASCII. Your output from the lab will be different because you have different plaintext, nonce, and key for the actual part01 lab exercise.

```
jrursch@ames-desktop-2:/mnt/3310_labs/labs/f25_labs/lab07/part01$ xxd -b part01_ciphertext.bin
00000000: 01001100 01000010 11000011 00100101 01101111 11010110  LB.%o.
00000006: 10010110 01110010 00011001 10110011 00000000 01010110  .r...V
0000000c: 11111100 10101001 11111011 11100000 10001010 00010000  ....
00000012: 00011011 10100111 11001010 10001011 00011111 00100110  ....&
00000018: 11101110 00000011 00001100 00111110 00110010 01101000  ...>2h
0000001e: 11110010 00011000 11011000 11101110 11101000 10000000  ....
00000024: 10001101 01110101 00010110 00010000 10001110 00111111  .u...?
0000002a: 10101111 11000010 00100011 00001010 01110011 01100011  ..#.sc
00000030: 10000010 10011001 00110111 00110010 00111000 01000010  ..728B
00000036: 01110110 00010110 01001001 11111000 11001000 10011101  v.I...
0000003c: 11000111 11010110 11000111 01111011  ...{
```

Part 02 - Decrypt - GCM (current)

5. Now you will decrypt `part02_ciphertext.bin`. You need to complete the TODO sections of the program for the decrypt function if you have not already done so to check your encryption function.
6. Again, you will find <https://cryptography.io/en/latest/hazmat/primitives/aead/#cryptography.hazmat.primitives.ciphers.aead.AESGCM> useful.
 - a. Use AESGCM, 128 bit keys
7. For this part, you received the message from Dr. Rursch. This means you'll have to change the AEAD for it to decrypt properly.
8. The syntax to run your decryption program is `python3 part01_02_aes_student_skel.py decrypt part02_key.bin part02_nonce.bin part02_ciphertext.bin part02_plaintext_out.txt`
9. The image below is an example. Your plaintext output will be different. **Include a screenshot of the result in your lab report.**

```
jrursch@ames-desktop-2:/mnt/3310_labs/labs/f25_labs/lab07/part02$ python3 ../part01/part01_02_aes_student_skel_jar.py d
crypt part02_key.bin part02_nonce.bin part02_ciphertext.bin part02_plaintext_out.txt
File 'part02_ciphertext.bin' decrypted successfully to 'part02_plaintext_out.txt'.
Decrypted content (ASCII): In my experience, there's no such thing as luck.
```

10. **Upload your code to Canvas.**
11. In our implementation, I am giving everyone the same symmetric encryption key and nonce. **This is inherently wrong. Why?**
12. **What difficult problem about using symmetric keys does this demonstrate? Think about the two parties (you and me) having to share the same key. What protocol have we started to discuss in lecture that can solve this problem?**

Part 03 - Old Block Cipher Modes

In this section of the lab, you will explore a problem AES encryption had when using ECB mode (Electronic Codebook). You will compare it to another older cipher and CBC (Cipher Block Chaining). Both of these block cipher modes are no longer used, for good reason which you will see in Part 03 and Part 04.

This is not steganography. BMP files have a clear structure, including a header and pixel data. When encrypted, the header must remain intact so that the image can still be opened by image viewers. This allows us to see the effect of encryption on the image data, especially how different modes either preserve or hide patterns in the image.

13. Use `part03_ecb_cbc_skel.py`. You will only modify the `encryptedEcb` and `encryptedCbc` functions. Again, use <https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/> to determine what values should be included.
14. The syntax is `python3 part03_ecb_cbc_skel.py part03_input.bmp part03_output_ecb.bmp part03_output_cbc.bmp`
15. **Open the two images and take a screenshot of them.** Include the image in your lab report.
16. Images often contain repetitive structures or patterns (e.g., large areas of the same color). **How does ECB mode handle these repetitive blocks, and why does this lead to visible patterns in the encrypted image?**

17. What effect does CBC mode have on repetitive structures in the image compared to ECB mode? How does the introduction of the Initialization Vector (IV) in CBC mode help in obscuring patterns that are visible in ECB mode?
18. Upload python code

Part 04 - A Flaw with CBC - the Padding Oracle

In this part of the lab, you are going to use a simulation of the padding oracle problem that is associated with using CBC. While our simulation runs on a local machine and demonstrates how the padding oracle works, it is a real vulnerability that was exploited in the wild.

When this vulnerability exists on a networked server, an attacker submits many modified ciphertexts and observes the server's external behavior (different HTTP status codes, error messages, or timing differences). Those varying responses act as the "oracle" that reveals whether the padding was valid. Over many queries an attacker can recover plaintext bytes one at a time.

To prevent this class of attack, we no longer use unauthenticated CBC with observable error messages. Instead we use an AEAD mode (e.g., AES-GCM or ChaCha20-Poly1305), or apply encrypt-then-MAC and verify the MAC before attempting decryption. Also returning a single generic error on failures and rate-limit/sensor suspicious requests reduces the oracle problem.

The script you will run generates a random AES key and IV, PKCS#7-pads a sample plaintext, and encrypts it with AES-CBC. It provides an automated byte-by-byte attack to recover plaintext blocks by repeatedly probing the local oracle.

19. Run the fully provided python script using the command
`python3 part04_padding_oracle_demo_v2.py`
20. In your own words, explain the CBC decryption occurring on the screen using the decryption notation from class. Note: $B_0 = IV$, $B_1 = C_0$, $B_2 = C_1$, etc.

$$P_0 = IV \oplus D(C_0, K),$$

$$P_1 = C_0 \oplus D(C_1, K),$$

$$P_2 = C_1 \oplus D(C_2, K), \dots$$

You will get an additional line of information in each of the Attacking block index X sections. It is the hex value derived from the padding oracle.

Please make a screenshot of your output and include it in the lab report. We need this since I used random values for the IV and key so each of your outputs will be different.

Then create a table where you complete the following. I am expecting 6 entries in the table with four lines for each of block 0, block 1, block 2, etc.

B0

Variable from slides	Corresponding variable from python output B0, B1, etc.	Hex value
IV		
(C_0, K)		
$D(C_0, K)$		
P		

B1 and so on

Variable from slides	Corresponding variable from python output B0, B1, etc.	Hex value
IV		
(C_0, K)		
$D(C_0, K)$		
P		

21. **Why can we recover the plaintext with the CBC flaw?**

Lab 07 Template

Part 01

1. **Print the `part01_ciphertext.bin` file and include a screenshot of the result in your lab report. Use `xxd -b <outputFile>`**
(5 points total)

Part 02

2. **Include a screenshot of the result of decrypting `part02_ciphertext.bin`.**
(5 points total)
3. **Functional code for both encrypt (part01) and decrypt (part02) submitted to Canvas as `<netid>_part01_02_aes.py`**
(10 points total)
4. **Why is giving everyone the same symmetric encryption key and nonce inherently wrong?**
(10 points total)
5. **What difficult problem about using symmetric keys does this demonstrate? Think about the two parties (you and me) having to share the same key. What protocol have we started to discuss in lecture that can solve this problem?**
(10 points total, 5 points each questions)

Part 03

6. **Open the two images and take a screenshot of them.**
(10 points)
7. **How does ECB mode handle these repetitive blocks, and why does this lead to visible patterns in the encrypted image?**
(10 points total, 5 points each question)
8. **What effect does CBC mode have on repetitive structures in the image compared to ECB mode? How does the introduction of the Initialization Vector (IV) in CBC mode help in obscuring patterns that are visible in ECB mode?**
(10 points total, 5 points each question)
9. **Upload python code for part 03.**
(10 points)

Part 04

10. **In your own words, explain the CBC decryption occurring on the screen using the decryption notation from class.**
Please make a screenshot of your output and include it in the lab report. We need this since I used random values for the IV and key so each of your outputs will be different.

Then create a table where you complete the following.

(10 points)

Variable from slides	Corresponding variable from python output B0, B1, etc.	Hex value
IV		
(C_0, K)		
$D(C_0, K)$		
P		

11. **Why can we recover the plaintext with the CBC flaw?**
(10 points)